Evaluating a Microservices Architecture for Scalable Multi-Agent AI Systems

Teammates: Francis, Luochun Xiao, Xinyue Xuan, Xuetao Li

Problem

Multi-agent AI applications often combine diversified tasks with different resource requirements and execution times. one waits on an LLM, one is I/O-bound on search, one burns CPU running code.

A monolithic deployment 1. limits independent scaling, 2. pays for idle parts to relieve one bottleneck.

Splitting into microservices allows independent scaling, but introduces network overhead, coordination complexity, and new failure modes.

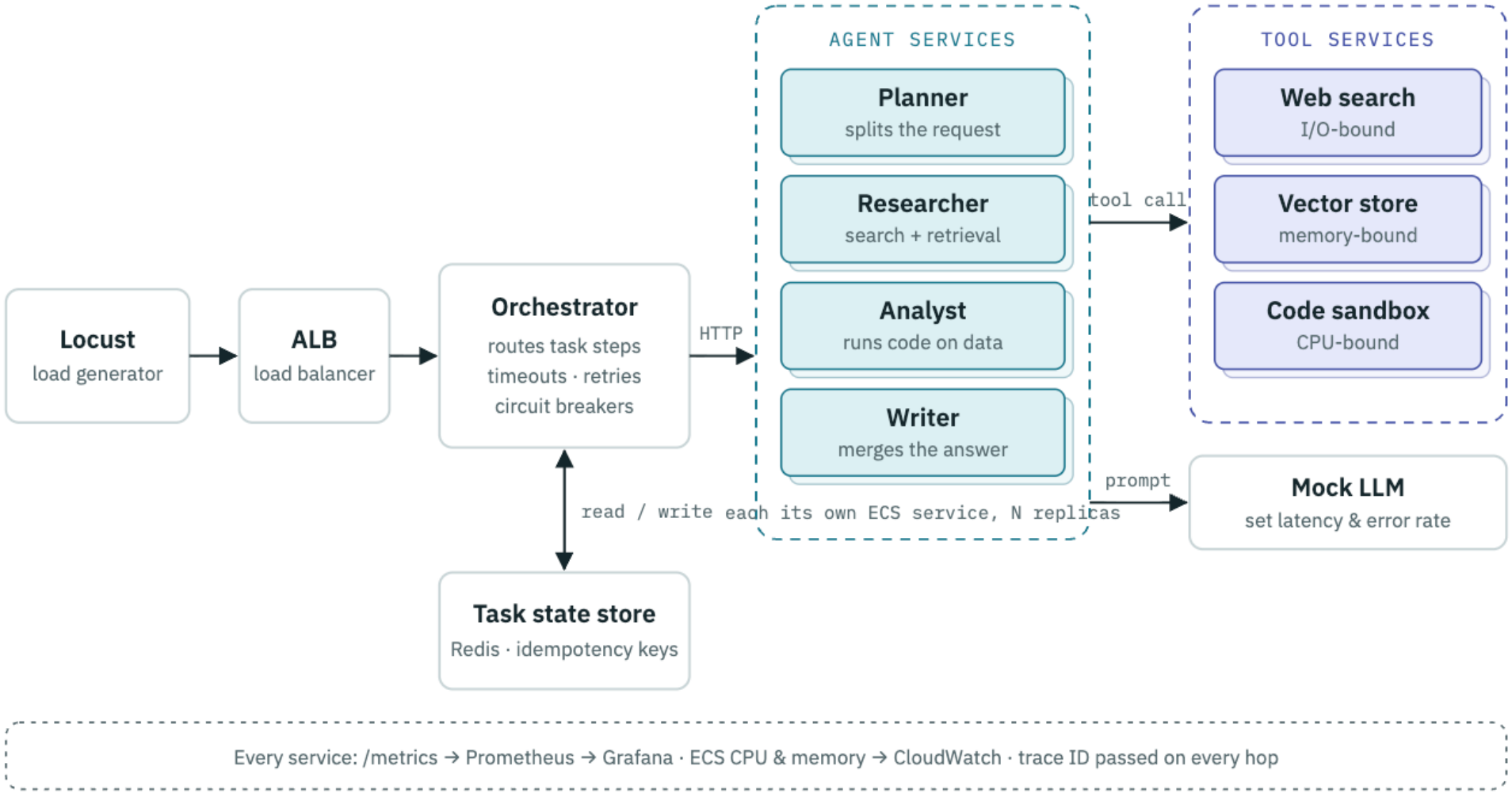
Stakeholders:

- Cost owners
- End users
- Teams building agent products
- Platform and SRE

Questions

1. How does microservices decomposition affect throughput and tail latency as workload increases?
2. How effectively do microservices architecture and failure-isolation mechanisms improve system performance and resilience?

Every agent and tool is its own service on AWS ECS



Experiment 1.A — Architecture Comparison

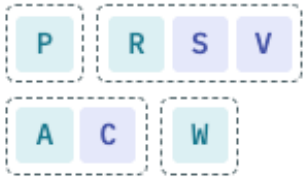
Same code, Four ways to draw the service boundaries

Monolith baseline



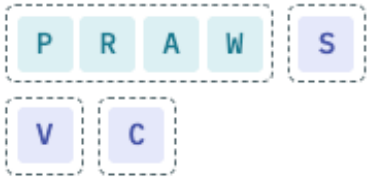
internal hops per research task **0**

Agents as services



internal hops per research task **3**

Tools as services



internal hops per research task **2**

Agents and tools



internal hops per research task **5**

P agent S tool one container / ECS service

Load: Locust steps up concurrent users (for example 10 → 400) and holds each step for a few minutes. Same task mix, same total vCPU budget for every variant, 3 repeats per step.

Metrics: throughput, p50/p95/p99 latency, CPU and memory per service, and the saturation point: the highest load where p99 stays under the SLO (e.g. 5 s) and errors stay under 1%.

Experiment 1.B — Scaling strategy

Scale the entire monolith versus scaling only the bottleneck microservice.



We first find the bottleneck from Experiment 1A (red outline), then compare both strategies under the same load. We also run ECS target-tracking autoscaling on CPU, as in homework 6, to see how fast each strategy reacts to a spike.

Metrics: peak throughput, total vCPU and memory used, throughput per vCPU, cost per 1,000 tasks, and time from scale-out decision to the new task serving traffic.

Experiment 2: Fault tolerance and failure isolation

Deliberately crash or slow down some agents in the system.

Setup: monolith vs. the fully decomposed variant

Metrics: Failure rate, p95 end-to-end latency, recovery time, task lost rate

MECHANISM	PROTECTS AGAINST
Timeouts	callers hanging on a slow service
Retry + backoff + jitter	brief crashes, retry storms
Circuit breaker	repeated calls to a dead service
Bulkhead (per-service limits)	one dependency exhausting all workers
Queue + idempotency keys	lost or duplicated tasks on retry
Health checks + ECS restart	crashed tasks staying down

Observation

Locust

Grafana

Cloudwatch to monitor CPU and storage metrics

Server-side logs for details